Lab9

Course: ARTI 404 – Image Processing

Student Name: Muhannad Almutlaq

Student ID: 2240006060

Procedural Task#1

Apply morphological erosion to Erosion.png. Load the image in grayscale and binarize it with Otsu's method (cv2.THRESH_BINARY + cv2.THRESH_OTSU) so the threshold is chosen automatically. Define three all-ones square structuring elements of size 3x3, 7x7 and 11x11 and apply cv2.erode with each one (iterations=1). Erosion shrinks the white foreground objects and removes small white specks and thin protrusions; the larger the kernel (or the more iterations), the more aggressive the erosion. Show the original binary image and the three eroded results in a 2x2 grid.

```
# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)

# 2. Binarize the image (Otsu's method works well for varying light)
_, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3. Define a 3x3 kernel (all ones), you can define kernel of size 5x5, 7x7, ... any
odd x odd
# Larger kernels lead to more aggressive erosion

kernel3 = np.ones((3, 3), np.uint8)
kernel7 = np.ones((7, 7), np.uint8)
kernel11 = np.ones((11, 11), np.uint8)

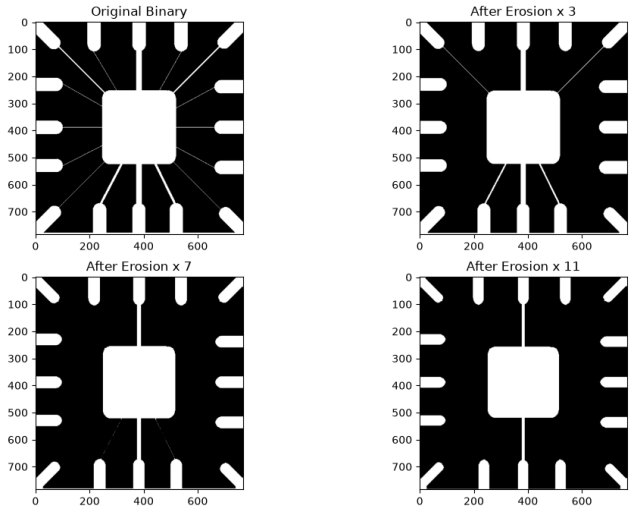
# 4. Apply erosion
# 'iterations' determines how many times the operation is repeated,
# ===> You can change iterations values to apply same process multiple times.

eroded_img3 = cv2.erode(binary, kernel3, iterations=1)
eroded_img7 = cv2.erode(binary, kernel7, iterations=1)
eroded_img11 = cv2.erode(binary, kernel11, iterations=1)

# 5. Display comparison
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1), plt.imshow(binary, 'gray'), plt.title('Original Binary')
plt.subplot(2, 2, 2), plt.imshow(eroded_img3, 'gray'), plt.title('After Erosion x 3')
plt.subplot(2, 2, 3), plt.imshow(eroded_img7, 'gray'), plt.title('After Erosion x 7')
```

```
plt.subplot(2, 2, 4), plt.imshow(eroded_img11, 'gray'), plt.title('After Erosion x
11')


plt.show()
```



Procedural Task#2

Apply morphological dilation to the same image. Load Erosion.png in grayscale, binarize it with Otsu's method, and define the same 3x3, 7x7 and 11x11 all-ones kernels. Apply cv2.dilate with each kernel (iterations=1). Dilation is the opposite of erosion: it grows the white foreground objects, fills small holes and gaps, and joins objects that are close together, and larger kernels make the effect stronger. Show the original binary image and the three dilated results in a 2x2 grid.

```
# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)

# 2. Binarize the image (Otsu's method works well for varying light)
_, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3. Define a 3x3 kernel (all ones), you can define kernel of size 5x5, 7x7, ... any
odd x odd
# Larger kernels lead to more aggressive erosion

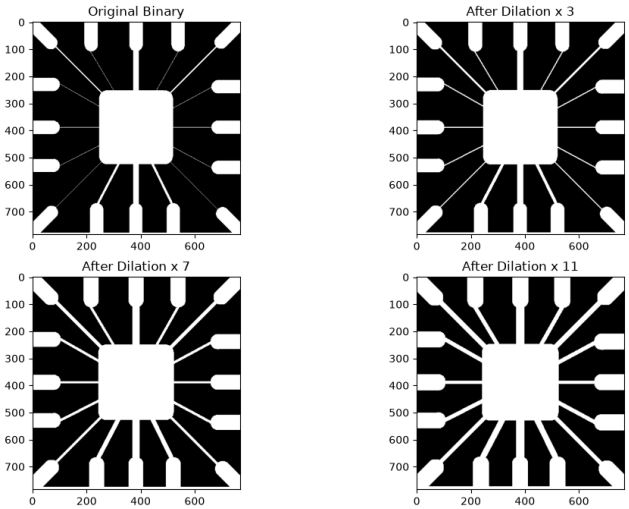
kernel3 = np.ones((3, 3), np.uint8)
kernel7 = np.ones((7, 7), np.uint8)
kernel11 = np.ones((11, 11), np.uint8)

# 4. Apply erosion
# 'iterations' determines how many times the operation is repeated,
# ===> You can change iterations values to apply same process multiple times.

eroded_img3 = cv2.dilate(binary, kernel3, iterations=1)
eroded_img7 = cv2.dilate(binary, kernel7, iterations=1)
eroded_img11 = cv2.dilate(binary, kernel11, iterations=1)

# 5. Display comparison
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1), plt.imshow(binary, 'gray'), plt.title('Original Binary')
plt.subplot(2, 2, 2), plt.imshow(eroded_img3, 'gray'), plt.title('After Dilation x 3')
plt.subplot(2, 2, 3), plt.imshow(eroded_img7, 'gray'), plt.title('After Dilation x 7')
plt.subplot(2, 2, 4), plt.imshow(eroded_img11, 'gray'), plt.title('After Dilation x
11')


plt.show()
```


Original Binary
After Dilation x 3
After Dilation x 7
After Dilation x 11
0
100
200
300
400
500
600
700
0
200
400
600

Procedural Task#3

Apply morphological opening to remove noise. Load Open_Closing.png, binarize it with Otsu's method, and define a 5x5 all-ones kernel. Apply cv2.morphologyEx with cv2.MORPH_OPEN, which is equivalent to dilate(erode(binary)): the erosion removes small noise objects and thin connections, and the dilation then restores the size of the objects that survived. Show the original noisy binary image and the result after opening side by side.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

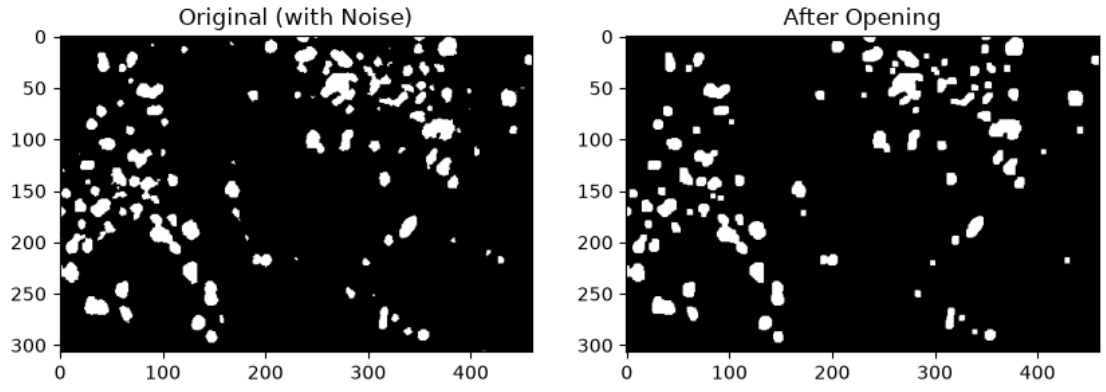
imgFile = 'Open_Closing.png'

# 1. Load image and binarize
img = cv2.imread(imgFile, 0)
_, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 2. Define kernel (a 5x5 square is a good starting point)
kernel = np.ones((5, 5), np.uint8)

# 3. Apply Opening
# This is equivalent to: dilate(erode(binary))
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

# 4. Display comparison
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1), plt.imshow(binary, 'gray'), plt.title('Original (with Noise)')
plt.subplot(1, 2, 2), plt.imshow(opening, 'gray'), plt.title('After Opening')
plt.show()
```



Procedural Task#4

Apply morphological closing to fill gaps and holes. Load Opening2.png, binarize it with Otsu's method, and define a large 45x45 all-ones kernel. Apply cv2.morphologyEx with cv2.MORPH_CLOSE, which is equivalent to erode(dilate(binary)): the dilation fills small holes, slots and notches in the foreground, and the erosion then restores the original size of the objects. Show the original binary image and the result after closing side by side.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

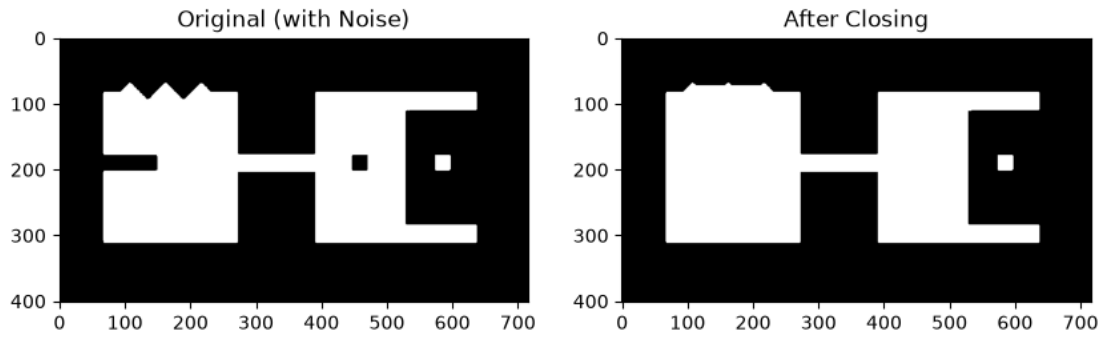
imgFile = 'Opening2.png'

# 1. Load image and binarize
img = cv2.imread(imgFile, 0)
_, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 2. Define kernel (a 45x45 square is a good starting point)
kernel = np.ones((45, 45), np.uint8)

# 3. Apply Opening
# Effectively: cv2.erode(cv2.dilate(binary, kernel), kernel)
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

# 4. Display comparison
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1), plt.imshow(binary, 'gray'), plt.title('Original (with Noise)')
plt.subplot(1, 2, 2), plt.imshow(closing, 'gray'), plt.title('After Closing')
plt.show()
```



Procedural Task#5

Extract object boundaries with morphological operations. Load BoundaryExtraction.png and binarize it with Otsu's method so that the foreground is white, then define a 3x3 kernel (a larger kernel gives thicker boundaries). Compute three boundary maps: the morphological gradient with cv2.morphologyEx and cv2.MORPH_GRADIENT (dilation minus erosion), the internal boundary (original minus eroded image), and the external boundary (dilated image minus original). Show the original image and the three boundary maps side by side.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt


imgFile = 'BoundaryExtraction.png'
# 1. Load and binarize the image (foreground must be white)
img = cv2.imread(imgFile, 0)
_, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 2. Define a kernel (the size controls boundary thickness)
kernel = np.ones((3, 3), np.uint8)

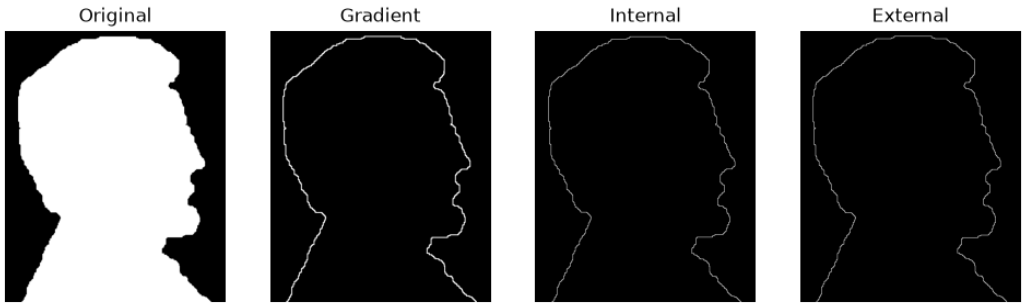
# 3. Method A: Morphological Gradient (Built-in standard)
gradient = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, kernel)

# 4. Method B: Internal Boundary (Original - Eroded)
eroded = cv2.erode(binary, kernel)
internal = cv2.subtract(binary, eroded)

# 5. Method C: External Boundary (Dilated - Original)
dilated = cv2.dilate(binary, kernel)
external = cv2.subtract(dilated, binary)

# Visualization
titles = ['Original', 'Gradient', 'Internal', 'External']
images = [binary, gradient, internal, external]

plt.figure(figsize=(12, 4))
for i in range(4):
    plt.subplot(1, 4, i+1)
    plt.imshow(images[i], 'gray')
    plt.title(titles[i])
    plt.axis('off')
plt.show()
```


Original
Gradient
Internal
External

Task#1

Apply the four basic morphological operations to Balls.png. Load the image in grayscale and binarize it with Otsu's thresholding. Define a standard 5x5 rectangular structuring element with cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5)), then apply erosion, dilation, opening and closing to the binary image. Display the original binary image and the four results side by side and compare them.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image and binarize using Otsu's Thresholding
image_path= "Balls.png"
original_image= cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

_, binary_image = cv2.threshold(original_image, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

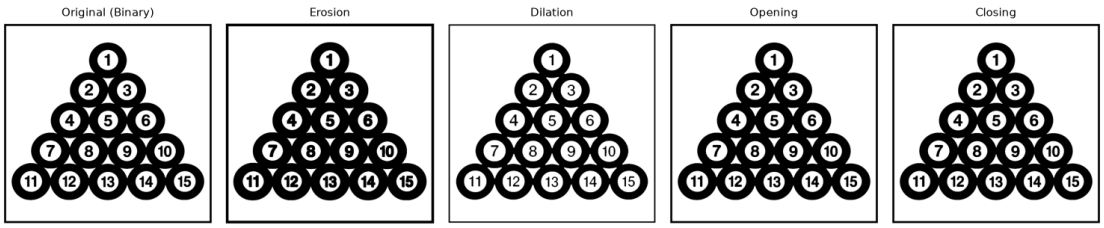
# Define a standard 5x5 rectangular structuring element
kernel= cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Try to apply morphological operations
eroded= cv2.erode(binary_image, kernel, iterations= 1)
dilated= cv2.dilate(binary_image, kernel, iterations= 1)
opened= cv2.morphologyEx(binary_image, cv2.MORPH_OPEN, kernel)
closed= cv2.morphologyEx(binary_image, cv2.MORPH_CLOSE, kernel)

# Display Results
titles= ["Original (Binary)", "Erosion", "Dilation", "Opening", "Closing"]
images= [binary_image, eroded, dilated, opened, closed]

plt.figure(figsize= (15, 6))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(images[i], cmap= "gray")
    plt.title(titles[i], fontsize= 11)
    plt.axis("off")

plt.tight_layout()
plt.show()
```



Observation: after Otsu's thresholding the white background is the foreground and the black balls are the background. Erosion therefore shrinks the white area, so the balls grow, the black rings get thicker and the white numbers get smaller and bolder. Dilation does the opposite: the balls shrink and the numbers become thinner. Because the image is clean and has no noise, opening and closing with a 5x5 kernel leave it almost unchanged compared with the original.

Task#2

Study the effect of the kernel shape and size on erosion and dilation. Binarize Balls.png with Otsu's method and test three structuring element shapes (cv2.MORPH_RECT for square, cv2.MORPH_CROSS for cross and cv2.MORPH_ELLIPSE for ellipse) with kernel sizes 3x3, 5x5 and 7x7. Create each kernel with cv2.getStructuringElement, apply cv2.erode and cv2.dilate with iterations=1, and plot all erosion and dilation results in a 3x6 grid (one row per shape) so the effects can be compared.

```
# Load image and binarize
image_path= "Balls.png"
original_img= cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

_, binary_img = cv2.threshold(original_img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

# Define shapes and kernel sizes to test
shapes= {
    "Square": cv2.MORPH_RECT,
    "Cross": cv2.MORPH_CROSS,
    "Ellipse": cv2.MORPH_ELLIPSE
}

kernel_sizes= [3, 5, 7]

# Iterate through shapes and sizes to compute and visualize results
fig, axes = plt.subplots(len(shapes), len(kernel_sizes) * 2, figsize= (18, 10))
fig.suptitle("Effect of Kernel Shape and Size on Erosion & Dilation", fontsize= 16,
fontweight= "bold")

for row_idx, (shape_name, shape_code) in enumerate(shapes.items()):
    for col_idx, k_size in enumerate(kernel_sizes):
        # Create kernel dynamically
        kernel= cv2.getStructuringElement(shape_code, (k_size, k_size))

        # try to apply erosion and dilation
        e_image= cv2.erode(binary_img, kernel, iterations= 1)
        d_image= cv2.dilate(binary_img, kernel, iterations= 1)

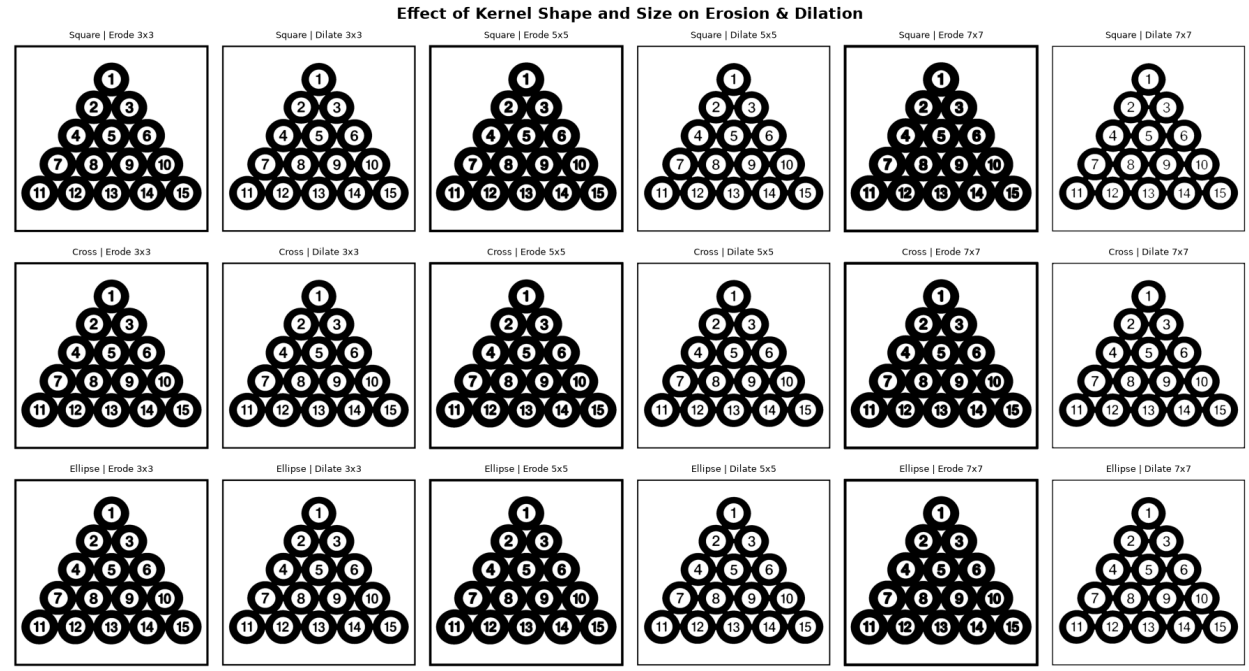
        # Column positioning for plot
        c_erode= col_idx * 2
        c_dilate= col_idx * 2 + 1

        # Plot erosion
        axes[row_idx, c_erode].imshow(e_image, cmap= "gray")
```

```
        axes[row_idx, c_erode].set_title(f"{shape_name} | Erode {k_size}x{k_size}",
fontsize= 9)
        axes[row_idx, c_erode].axis("off")

        # Plot dilation
        axes[row_idx, c_dilate].imshow(d_image, cmap= "gray")
        axes[row_idx, c_dilate].set_title(f"{shape_name} | Dilate {k_size}x{k_size}",
fontsize= 9)
        axes[row_idx, c_dilate].axis("off")

plt.tight_layout()
plt.show()
```



Observation: the larger the kernel, the stronger the effect. With 3x3 the results are close to the original, while with 7x7 erosion makes the black rings clearly thicker (the white numbers shrink) and dilation makes them clearly thinner. The differences between the three shapes are small on this clean image with round balls: the square kernel is the strongest, because it also acts along the diagonals, the cross kernel is the gentlest, and the ellipse kernel lies in between and gives the smoothest, most rounded result.